
When Does Monitoring Long-Horizon Agents Lead to Better Performance?

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Long-horizon language-model agents can experience performance degradation
2 as their interaction histories grow and context accumulates in their agent state
3 [4]. We study how well different monitoring methods predict agent degradation,
4 and whether compacting or reconstructing agent state improves downstream task
5 performance. Across conversational reasoning tasks (Evolving-Intent GSM8K
6 [9]) and multi-turn tool-use tasks (BFCL Multi-Turn [5]), we compare “active”
7 observational monitors, “passive” monitors, and baseline context- and time-based
8 approaches. We find that monitoring is not a free observation: active probes can
9 alter the trajectories they are intended to measure, producing an “observer effect”
10 where task success tends to decrease. We further find that the value of monitoring
11 can depend on how agent state is recovered. In a controlled study, compaction and
12 deterministic reconstruction provide similar gains without a carried probe (+5.9
13 and +6.3 percentage points). Carrying the probe reduces the gain under compaction
14 to +2.8 points, while the gain under reconstruction remains +6.5 points. These
15 results suggest a taxonomy in which observation method, signal quality, and
16 recovery mechanism are evaluated jointly, rather than treating monitoring as a
17 standalone prediction problem.

18 1 Introduction

19 Modern agentic workflows have seen increasing adoption of tasks that involve multi-step tool use,
20 where they must maintain goals, constraints, and state over extended trajectories [2, 15]. As these
21 trajectories grow longer, however, agents can gradually lose important context and start hallucinating
22 incorrect information [3]. A natural response is to monitor an agent during execution and intervene
23 whenever its behavior starts to degrade. This can include restarting the agent, compacting its context,
24 or restructuring its state before the agent continues execution [7, 11, 16].

25 The need to preserve agent behavior creates an appealing motivation: if we can detect degradation
26 earlier and more accurately, we should be able to intervene at better times and improve task per-
27 formance. This has drastic implications for all agent tasks, whether it be coding benchmarks like
28 SWE-bench [2] or enterprise workflows like τ -bench [13].

29 Much of agent monitoring can thus be viewed as a prediction problem: given an agent’s current
30 behavior/state, determine whether it is likely to fail in the near future. Under this view, a more accurate
31 monitor should naturally produce a better agent, if better timing does indeed improve intervention.

32 Let M_t denote a monitor alarm at turn t , $F_{>t}$ a subsequent failure, $\pi(M)$ the policy that acts on the
33 alarm, and Y final task success. The usual prediction-first view assumes

$$(\Pr(M_t = 1 \mid F_{>t} = 1) \uparrow, \Pr(F_{>t} = 1 \mid M_t = 1) \uparrow) \implies \mathbb{E}[Y \mid \pi(M)] \uparrow. \quad (1)$$

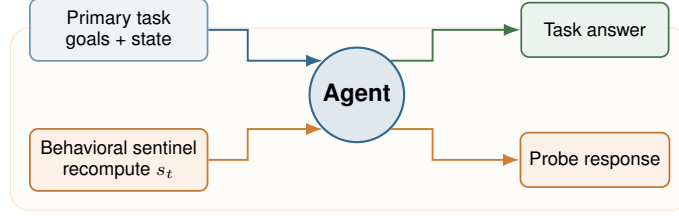


Figure 1: A behavioral sentinel is carried beside the primary task. Its response can warn of degradation, but the additional instruction and state also become part of the trajectory being measured.

We argue this view is incomplete; runtime monitors do not necessarily have to be a passive observer of an agent trajectory.

We explore a method of monitoring that introduces additional instructions or computations directly into the system prompt, separate from the original task we gave the agent.

We study one such class of monitors, which we call behavioral sentinels: canaries that an agent must maintain throughout a task and whose degradation can serve as a warning signal. For example, an agent might be asked to remember and update a small piece of state while simultaneously completing its primary task. The intuition here is if the agent fails to maintain the state before completing the underlying task, the sentinel can provide an early warning signal.

However, asking the agent to maintain such a signal can itself change the trajectory being measured. Across our experiments, we find an “observer effect”: a probe tends to alter the very trajectory it is responsible for measuring. Monitoring therefore is not always a free source of information, and a higher quality signal does not necessarily produce a better agent.

This raises a second question. If active monitoring changes the agent state, does that change persist after intervention? The answer may depend on how the system recovers. When recovery relies on an agent-generated summary, monitoring state and task state must pass through the same lossy reconstruction process. External reconstruction instead rebuilds state from records maintained by a harness outside the agent’s context. We therefore test whether the cost of carrying an active monitor changes with the recovery mechanism used after intervention.

Our experiments reveal that under lossy-agent generated compaction, carrying additional state can reduce the benefit of an otherwise well-timed intervention. However, when the state is reconstructed through deterministic reconstruction, this penalty largely disappears and the monitoring setup can be beneficial.

Our work makes three primary contributions:

- We show that active monitoring is not a free observation: behavioral probes can alter the trajectories they are intended to measure.
- We isolate how carried monitoring state interacts with recovery. Holding the monitoring condition fixed, we test whether its cost changes when recovery shifts from lossy agent-generated compaction to deterministic external re-grounding.
- We evaluate monitoring as a complete systems problem. Across multiple models and long-horizon tasks, we find that no single monitoring strategy consistently dominates.

2 Related Works

2.1 What has already been done

Recent work has explored detecting degradation before task completion

Doomed from the Start [6] leverages probes over internal model activations to predict task failure and terminate struggling trajectories. *TACT* [8] similarly detects agent drift from internal activation and uses activation steering to mitigate it during execution.

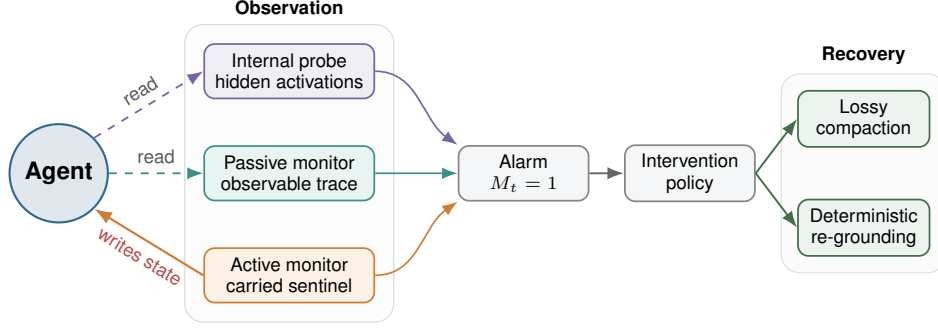


Figure 2: Three monitoring methods and their relationship to the agent. Internal probes read hidden activations, passive monitors read the observable trace, and active monitors write additional state into the trajectory. Each signal ultimately controls an intervention and recovery mechanism.

71 Other work has introduced explicit diagnostic probes: *Canary Tools* [1], for example, inserts adver-
72 sarially designed tools into an agent’s context to expose weaknesses in tool-selection reasoning.

73 *Accurate Failure Prediction in Agents Does Not Imply Effective Failure Prevention* [10] further shows
74 that accurate prediction of failure by itself does not imply effective prevention.

75 *Agentic Uncertainty Quantification* [14] takes a complementary approach by estimating uncertainty
76 throughout an agent trajectory and using it to control uncertainty-aware memory and targeted
77 reflection. This couples detection and intervention, while leaving open how the monitoring mechanism
78 and post-intervention state representation jointly affect end-to-end performance, which we explicitly
79 address.

80 Together, these methods show us that agent failure can often be detected before task completion,
81 whether it be internal representations, observing trajectories, or introducing probes, but whether that
82 leads to genuine performance improvement is still unclear.

83 2.2 What gap remains

84 A newer line of work explores what should happen after degradation is detected. *Fail-Fast, Restart-*
85 *Smart* [11] combines early failure prediction with different restart strategies, showing us that pre-
86 serving useful work across sessions can outperform a cold restart. *AgentRewind* [16] further treats
87 recovery as a state management problem, checkpointing both agent context and environment state, so
88 execution can return to an earlier point after agent failure.

89 Separate work on long-horizon context management has noted that LLM-generated compaction is
90 inherently lossy and can unpredictably discard important information that is needed for later sessions
91 [7].

92 2.3 Why this paper is different

93 Prior work therefore establishes two notions: agent failures can be monitored and that choosing
94 the right recovery strategy matters. However, these components are typically evaluated separately.
95 Monitoring work primarily asks whether a signal predicts a failure, while recovery work asks how
96 best to restore or restart an agent once intervention occurs. To our knowledge, prior work has not
97 directly isolated whether the cost of carried active monitoring changes with the recovery mechanism
98 used to reconstruct agent state. We study this precise interaction explicitly. Using behavioral sentinels
99 and passive monitors, we measure both signal quality and how observation changes task execution
100 and recovery.

101 In particular, we hold the monitoring setup fixed while replacing lossy agent-generated compaction
102 with deterministic external re-grounding, allowing us to precisely test whether the effect of monitoring
103 changed depending on the recovery mechanism.

Table 1: Controlled 2×2 mechanism study.

	No intervention	Oracle-timed intervention
No carried probe	<i>A</i>	<i>B</i>
Carried probe	<i>C</i>	<i>D</i>

3 Experimental Setup

Our experiments follow a two-stage design. We first use a controlled study to isolate how monitoring, intervention timing, and recovery interact. We then evaluate whether these conclusions persist in a deployment-oriented study using larger models and realistic agent benchmarks.

Across both studies, we separate three components of an agent system:

Monitoring strategy: what signal is used to detect degradation?

Intervention policy: when does the system act on that signal?

Recovery mechanism: how is agent state reconstructed after intervention?

3.1 Controlled mechanism study

We first study the monitoring recovery interaction under controlled conditions using GPT-OSS-20B on long-horizon coding, register-tracking, and bAbI-style tasks [12]. These environments require intermediate progress to persist in agent state across multiple turns and allow us to identify when task performance begins to degrade.

Our primary active monitors are behavioral sentinels that require information that cannot simply be copied from the model’s previous response. Accordingly, we construct probes whose correct response must be recomputed from the current state. For example, a probe can require an arithmetic check derived from values in the current turn. Failure to maintain the sentinel provides a possible signal of underlying degradation.

Because these probes are carried directly inside the agent’s context, observing the agent may itself change the trajectory being measured. We therefore compare otherwise identical trajectories with and without a carried probe.

Intervention timing. To separate monitor quality from the value of acting at the correct time, we compare trajectories with no intervention against trajectories receiving an oracle-timed intervention. Oracle timing uses knowledge of subsequent task degradation and is not intended as a deployable monitor. Instead, it provides a controlled estimate of the performance headroom available from favorable intervention timing.

To create the Oracle ground truth for our coding tasks, we retrospectively use a deterministic checker to label each turn as correct or hallucinated. The checker parses the agent-emitted Python code and compares each requested function definition and reported signature against the task’s authoritative symbol table; it also detects invalid syntax, missing required definitions or calls, invented symbols, and incorrect claims that a function exists or was deleted. A turn is labeled correct only if none of these checks finds an error. The oracle intervenes immediately before the first turn labeled as hallucinated. If no hallucination occurs, it does not intervene.

We repeat the intervention conditions under two recovery mechanisms:

Agent-generated compaction asks the model to compress its accumulated context into a shorter representation before continuing execution. Because recovery depends on the model’s existing trajectory, information that has already been forgotten, distorted, or deprioritized may not survive compaction.

External reconstruction instead regrounds the relevant task state deterministically from an external source of truth. For example, a register-tracking task can replay the authoritative user-issued updates to rebuild every current register value. It does not depend on what information the model itself retained immediately before intervention.

146 Before running the reconstruction condition, we pre-specified our prediction and fixed the task set,
 147 thresholds, exclusions, and sample size. We predicted that external reconstruction would reduce the
 148 additional cost of carrying the probe.

149 Because no recovery occurs in conditions A or C , these conditions are shared across operators. The
 150 recovery mechanism affects only the intervention conditions B and D .

151 For recovery operator R , we measure whether carrying a probe changes the value of an otherwise
 152 well-timed intervention:

$$\Delta_{\text{interaction},R} = [P(D | R) - P(C)] - [P(B | R) - P(A)]. \quad (2)$$

153 Intuitively, this comparison asks whether carrying the same probe changes the benefit of intervention
 154 differently under compaction than under reconstruction.

155 3.2 Deployment-oriented evaluation

156 The controlled study is designed to isolate mechanisms, but it does not capture the full conditions
 157 of deployed agent monitoring. We conduct a second set of experiments on BFCL Multi-Turn and
 158 Evolving Intent GSM8K using DeepSeek V4 Flash, GPT-OSS-120B, GPT-5.6 Luna, and GPT-5.6
 159 Terra.

160 Here, monitors determine when intervention occurs without oracle timing. We distinguish between
 161 monitors that directly interact with the agent during execution (“active”) and monitors that observe
 162 the trajectory without modifying it (“passive”).

163 Active monitors introduce additional instructions or state directly into the agent’s context. Passive
 164 or zero-carry monitors obtain a signal without persisting additional monitoring state in the agent’s
 165 trajectory. These include trace-based monitoring and forked evaluation strategies.

166 We evaluate these monitoring strategies under three post-intervention conditions:

- 167 1. No intervention
- 168 2. LLM-generated context compaction
- 169 3. Deterministic reconstruction

170 In our study, the “compaction” intervention approach asks the agent to summarize its current stored
 171 state in a more succinct form. We then erase the stored state and continue from the task instructions
 172 and this model-written summary.

173 The “reconstruction” approach also starts a fresh trace, but it rebuilds state from records maintained by
 174 a harness outside the agent’s context. These records contain only the original instructions and public
 175 task information observed so far, not the agent’s memory, and encourages the agent to re-retrieve the
 176 context it needs from its workspace.

177 Unlike the controlled mechanism study, monitoring methods act at their natural firing times in this
 178 setting. Intervention rates are not forced to match across methods, although each policy is limited to
 179 at most one intervention per task. We report intervention rates alongside task success to distinguish
 180 differences in recovery quality from differences in how frequently a monitoring policy fires.

181 3.3 Evaluation metrics and protocol

182 We evaluate each monitoring system at two levels.

183 **Monitoring quality.** We report precision, recall, and area under the precision-recall curve (AUPRC)
 184 to measure how well a signal identifies subsequent task degradation.

185 **End-to-end utility.** We measure final task success, intervention frequency, token usage, latency,
 186 and monetary cost where available. These metrics capture whether a monitoring signal actually
 187 improves the complete agent system rather than just predicting failure accurately.

Active observation usually reduces task success

Paired active-recompute minus clean-arm success; whiskers are task-bootstrap 95% intervals

Within each benchmark, models are sorted from the largest decrease to the largest increase

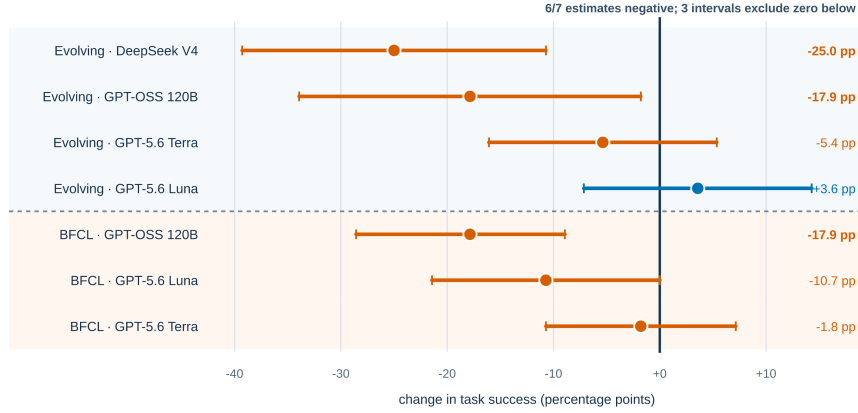


Figure 3: Observer effect for the prespecified active-recomputation contrast. Points are paired changes in task success relative to a clean arm on the same 56 tasks; whiskers are task-bootstrap 95% intervals.

4 Results

4.1 No monitoring method consistently provides the best signal

We first compare how well each monitoring method predicts agent degradation. Adjusted for firing rates, no single method consistently performs best across models and tasks. Active recomputation achieves the highest AUPRC in three of seven model-task settings, while passive or baseline methods lead in the remaining four.

The strongest individual signal occurs on BFCL with GPT-OSS-120B, where active recomputation reaches an AUPRC of 0.942. However, this monitor also illustrates an important tradeoff between precision and recall: it achieves a precision of 1.00 but a recall of only 0.20. Its alarms are highly reliable when they occur, but most failures occur without an alarm.

Overall, signal quality is highly setting-dependent. A monitoring strategy that performs well for one model or benchmark does not necessarily remain the strongest in another.

4.2 Active monitoring can change the trajectory it observes

Signal quality alone does not capture the cost of obtaining that signal. We therefore compare active-recomputation trajectories with clean baseline trajectories to determine whether introducing the probe changes task performance.

Six of seven settings show a negative change in task success when active monitoring is introduced, with three decreases statistically significant. The largest effect occurs for DeepSeek V4 on Evolving Intent, where success decreases by 25.0 percentage points. GPT-5.6 Luna on Evolving Intent is the only setting with a positive point estimate, increasing by 3.6 points, although this effect is not statistically significant.

These results provide evidence of an “observer effect”: active probes largely negatively impact the trajectories they are intended to measure. This creates a distinction between a monitor’s predictive quality and its actual effect on the agent. A monitor may provide useful information about degradation while simultaneously changing the probability that the underlying task succeeds.

4.3 The cost of carried monitoring state can depend on recovery

We next ask whether carrying an active monitor changes how much value the system can obtain from a well-timed intervention.

Perfect timing helps, but recovery determines how much

Controlled coding factorial: GPT-OSS-20B; $n = 100$ tasks

Oracle resets immediately before the first hallucinated turn in the matched no-intervention trajectory

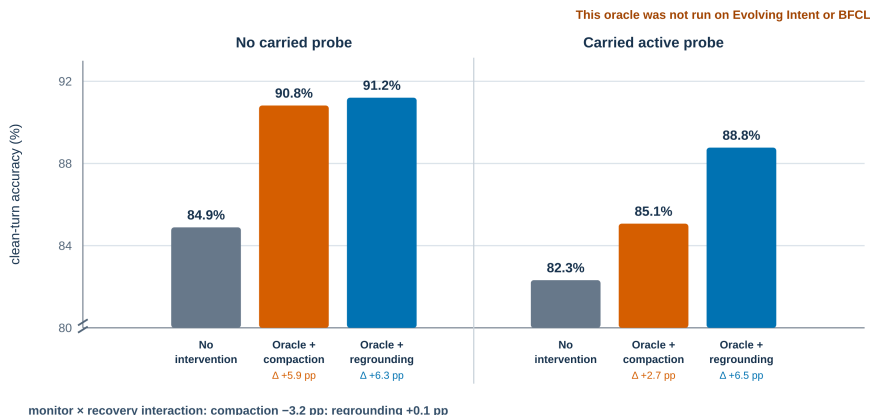


Figure 4: Oracle-timed intervention in the controlled coding factorial ($n = 100$ tasks). The oracle resets immediately before the first hallucinated turn from the matched no-intervention trajectory. The result measures clean-turn accuracy and should not be read as an Evolving Intent or BFCL oracle comparison.

Without a carried probe, favorable intervention timing provides similar gains under both recovery mechanisms. Accuracy increases from 0.849 with no intervention to 0.908 with compaction and 0.912 with deterministic external reconstruction, corresponding to gains of 5.9 and 6.3 percentage points.

The pattern changes when the probe is carried. Under compaction, accuracy increases from 0.823 to 0.851, so the benefit of favorable timing falls to 2.8 percentage points. Under deterministic reconstruction, however, accuracy increases from 0.823 to 0.888, a gain of 6.5 points. Thus, while carrying the probe substantially reduces the observed timing benefit under compaction, the same reduction is not observed under reconstruction.

The corresponding interaction estimate is -3.2 percentage points (95% CI $[-7.5, +1.1]$) under compaction and $+0.1$ points (95% CI $[-4.1, +4.2]$) under reconstruction. Directly comparing these interactions gives a $+3.3$ percentage-point contrast (95% CI $[-0.3, +7.0]$), which tests whether carrying the probe changes the benefit of well-timed intervention differently across the two recovery mechanisms. The contrast is in the direction of our pre-specified prediction, although its confidence interval narrowly includes zero.

Importantly, deterministic external reconstruction tested a prediction specified before that condition was run: if the additional impact of the probe arose from interacting with lossy agent-generated summary, replacing compaction with external reconstruction can help reduce that penalty. The observed results are consistent with this prediction.

4.4 End-to-end performance depends on the monitor–recovery pair

Finally, we examine whether this dependence persists when monitors fire naturally rather than using oracle timing.

Across six monitoring methods on GPT-5.6 Luna and 40 Evolving Intent tasks, deterministic reconstruction achieves an average task success rate of 75.0%. This compares with 72.9% when monitoring triggers no state recovery and 59.6% under LLM-generated compaction.

The aggregate averages, however, hide substantial differences between monitoring methods. Active recomputation reaches 72.5% success when paired with compaction but 85.0% when paired with deterministic reconstruction. Other monitors do not follow the same ordering. Trace rules perform 5.0 percentage points better with compaction, Frozen recompute performs 2.5 points better with compaction, and Trace judge performs equally under the two recovery mechanisms.

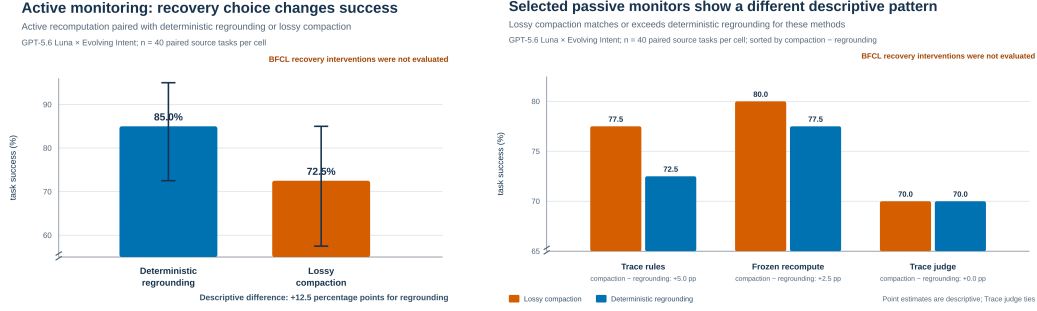


Figure 5: Recovery results in the GPT-5.6 Luna × Evolving Intent deployment ($n = 40$ paired source tasks per cell). *Left*: active recomputation under lossy compaction and deterministic reconstruction, with task-bootstrap 95% intervals. *Right*: selected passive monitors under the same two operators. Frozen quiz is excluded, firing rates are natural rather than matched, and BFCL recovery interventions were not evaluated.

The deployment results therefore do not support a single globally best monitor or recovery strategy. Instead, performance depends on how the two are paired. A recovery mechanism that improves one monitoring strategy may provide little benefit, or even perform worse, with another.

It is important to note, however, monitoring methods fire at their natural rates, so intervention frequency is not matched across conditions, although each policy is limited to at most one intervention per task. The study is also limited to 40 Evolving Intent tasks and is sensitive to task inclusion. We therefore treat the exact ranking of individual monitor-recovery pairs as preliminary, while using the controlled experiment above as the stronger evidence for the monitoring-recovery interaction.

5 Discussion and Conclusion

Our results suggest that runtime monitoring should be evaluated not only by the signal it produces, but by how observation changes agent state and whether the recovery mechanism preserves or removes those changes. Predictive metrics such as precision, recall, and AUPRC describe how well a monitor identifies degradation, but they do not capture what the monitor changes while observing the agent or what happens after its signal triggers an intervention. In our experiments, both of these factors matter. An important open question is what determines whether an active monitor hurts, helps, or leaves task performance unchanged. Properties such as model capability, task structure, context pressure, and the amount of additional monitoring state may all play a role.

Several limitations in our experiments motivate further study. Our cleanest controlled evidence comes from synthetic long-horizon tasks, while the deployment recovery study is limited to GPT-5.6 Luna on 40 Evolving Intent tasks. Monitoring methods also fire at their natural rates rather than under matched intervention budgets, making the deployment comparisons descriptive rather than fully controlled. A key next step is therefore to reproduce the monitoring-recovery interaction on realistic coding and tool-use benchmarks across additional models while matching intervention frequency. Future work should also evaluate a broader spectrum of recovery mechanisms, including checkpoints, partial reconstruction, structured external memory, and hybrid approaches that combine model-generated summaries with externally maintained state.

Finally, our deterministic reconstruction experiments assume that the external state is reliable. Real agent systems may instead operate with stale, incomplete, or inconsistent state. Varying reconstruction quality would help determine when external reconstruction remains preferable to compaction and whether there is a threshold the benefits of monitoring begin to disappear.

More generally, understanding how observation cost, signal quality, intervention timing, and recovery accuracy interact can play an essential role in improving visibility of agent health. Advancements in observability can make agent steering systems more reliable for long-horizon tasks in enterprise and safety-critical settings, where agent degradation may directly affect people that depend on these services.

References

- [1] Atul Anand and Sourav Chattaraj. Diagnosing tool-selection reasoning in LLM agents with canary tools. *arXiv preprint arXiv:2608.04719*, 2026. URL <https://arxiv.org/abs/2608.04719>.
- [2] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VTF8yNQM66>.
- [3] Philippe Laban, Hiroaki Hayashi, Yingbo Zhou, and Jennifer Neville. LLMs get lost in multi-turn conversation. *arXiv preprint arXiv:2505.06120*, 2025. URL <https://arxiv.org/abs/2505.06120>.
- [4] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. doi: 10.1162/tac1_a_00638. URL <https://aclanthology.org/2024.tac1-1.9/>.
- [5] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 48371–48392. PMLR, 2025. URL <https://proceedings.mlr.press/v267/patil25a.html>.
- [6] Kai Ruan, Zihe Huang, Ziqi Zhou, Qianshan Wei, Xuan Wang, and Hao Sun. Doomed from the start: Early abort of LLM agent episodes via a recall-controlled probe cascade. *arXiv preprint arXiv:2607.06503*, 2026. URL <https://arxiv.org/abs/2607.06503>.
- [7] Andrew Semenov and Svyatoslav Dorofeev. Beyond compaction: Structured context eviction for long-horizon agents. *arXiv preprint arXiv:2606.11213*, 2026. URL <https://arxiv.org/abs/2606.11213>.
- [8] Yuan Sui, Yulin Chen, Yibo Li, Xue Jiang, Yufei He, Yihong Dong, Xiaoxin He, Tianyu Gao, and Bryan Hooi. TACT: Mitigating overthinking and overacting in coding agents via activation steering. *arXiv preprint arXiv:2605.05980*, 2026. URL <https://arxiv.org/abs/2605.05980>.
- [9] Jihoon Tack, Philippe Laban, and Jennifer Neville. LLMs get lost in evolving user intent. *arXiv preprint arXiv:2607.20734*, 2026. URL <https://arxiv.org/abs/2607.20734>.
- [10] Rakshith Vasudev, Melisa Russak, Dan Bikel, and Waseem Alshikh. Accurate failure prediction in agents does not imply effective failure prevention. *arXiv preprint arXiv:2602.03338*, 2026. URL <https://arxiv.org/abs/2602.03338>.
- [11] Chenyu Wang, Yunbo Lyu, Junda He, Zhou Yang, Chenxing Zhong, Yaniv Harel, and David Lo. Fail-fast, restart-smart: Early failure prediction and restart for SWE agentic tasks. *arXiv preprint arXiv:2608.03222*, 2026. URL <https://arxiv.org/abs/2608.03222>.
- [12] Jason Weston, Antoine Bordes, Sumit Chopra, Alexander M. Rush, Bart van Merriënboer, Armand Joulin, and Tomas Mikolov. Towards AI-complete question answering: A set of prerequisite toy tasks. *arXiv preprint arXiv:1502.05698*, 2015. URL <https://arxiv.org/abs/1502.05698>.
- [13] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=roNSXZpUDN>.
- [14] Jiaxin Zhang, Prafulla Kumar Choubey, Kung-Hsiang Huang, Caiming Xiong, and Chien-Sheng Wu. Agentic uncertainty quantification. *arXiv preprint arXiv:2601.15703*, 2026. URL <https://arxiv.org/abs/2601.15703>.

- 329 [15] Yuxiang Zheng, Dayuan Fu, Xiangkun Hu, Xiaojie Cai, Lyumanshan Ye, Pengrui Lu, and
330 Pengfei Liu. DeepResearcher: Scaling deep research via reinforcement learning in real-world
331 environments. In *Proceedings of the 2025 Conference on Empirical Methods in Natural*
332 *Language Processing*, pages 414–431. Association for Computational Linguistics, 2025. doi:
333 10.18653/v1/2025.emnlp-main.22. URL [https://aclanthology.org/2025.emnlp-main.](https://aclanthology.org/2025.emnlp-main.22/)
334 22/.
- 335 [16] Yu Zhuang, Kefei Chen, Yitong Duan, Shuxin Zheng, Jian Li, and Xu-Yao Zhang. AgentRewind:
336 Recoverable execution for long-horizon LLM agents. *arXiv preprint arXiv:2608.14380*, 2026.
337 URL <https://arxiv.org/abs/2608.14380>.